GRIT Interview Operations

AI Product Engineer - Round 1 | Decision-ready two-page product brief

A. Problem understanding and proposed solution

Problem. GRIT learners complete the Skills/Levels required for a Track Badge, then need a Level Clearance Interview. A small interviewer pool creates a capacity and coordination problem: eligibility, competing bookings, outcomes, retakes and later governance actions must remain consistent across three roles.

Solution. A role-based workspace gives students an eligibility-gated booking journey and status timeline; interviewers manage non-overlapping availability and submit PASS, FAIL or RETAKE LATER with feedback; Ops monitors the funnel, capacity and exceptions. The working local demo persists in the browser. The included Supabase/PostgreSQL migration is the production path with authorization, normalized records and transactional booking.

B. Interview lifecycle and transition permissions

From	Valid next state(s)	Actor / guard
NOT_ELIGIBLE	ELIGIBLE_NOT_BOOKED	System: every required skill/level verified
ELIGIBLE_NOT_BOOKED	BOOKED	Student via atomic booking RPC
BOOKED	COMPLETED_PENDING_RESULT; CANCELLED; NO_SHOW	Interviewer/Ops; student cancellation only per policy
COMPLETED_PENDING_RESULT	PASSED; FAILED; RETAKE_REQUIRED	Assigned interviewer or Ops
PASSED	BADGE_EARNED	System after committed result
FAILED	RETAKE_REQUIRED (if policy permits)	Ops-controlled exception
RETAKE_REQUIRED	RETAKE_BOOKED	Student via atomic booking RPC
RETAKE_BOOKED	COMPLETED_PENDING_RESULT; CANCELLED; NO_SHOW	Interviewer/Ops
CANCELLED	ELIGIBLE_NOT_BOOKED	System if still eligible
NO_SHOW	RETAKE_REQUIRED	Ops under configured policy
BADGE_EARNED	NULLIFIED	Authorized Admin, verified case only
NULLIFIED	None	Terminal; review handled outside state mutation

Permission invariant: students can request booking/cancellation only. They cannot write interview results, badge states or malpractice decisions. Every server mutation validates the current state against the allowlist to reject stale or invalid transitions.

C. Edge cases and assumptions

Scenario	Expected behavior
Same slot / double click	Row lock + unique slot + idempotency key: exactly one booking commits.
Ineligible or duplicate active booking	Server rejects; UI explains missing requirements or existing booking.
Interviewer withdraws booked time	Booking is preserved; Ops must cancel/reschedule and notify the learner.
Overlap / expired or blocked slot	PostgreSQL exclusion constraint and server time/status check reject write.
Cancel, reschedule, no-show	Cancellation releases slot when policy allows; reschedule is cancel + new atomic booking; no-show is authorized state transition.
Result missing	Pending-results queue; reminder and Ops escalation after configurable 24-hour SLA.
Retake	RETAKE LATER creates controlled eligibility window and a new attempt number.
No capacity / all full	Honest empty state; Ops workload view reveals capacity gap.
Refresh / timezone / wrong client clock	Read persisted server record; store UTC timestamptz and render explicit IST.
Malpractice allegation	Remain ALLEGED. Only verified Admin decision can nullify and create immutable history/audit.

Configurable assumptions: 60-minute interviews, IST display, cancellation cutoff, retake window/attempt cap, result SLA and appeal contact. These are not presented as existing NIAT policy.

GRIT Interview Operations

Data integrity, concurrency and product decisions

D. Normalized data model

Table	important fields / relationships / constraints
users	PK id; auth_user_id UNIQUE; role; email UNIQUE; created_at, updated_at
students / interviewers	PK/FK users.id; student_code UNIQUE; badge_id FK; lifecycle state; timezone; interviewer active
track_badges / required_skills	Badge PK; name UNIQUE; skill rows UNIQUE(badge_id, skill_code), required_level
student_skills	FK student; UNIQUE(student_id, skill_code); verified_level/at
interviewer_availability	FK interviewer; start/end; blocked; EXCLUDE overlapping tstzrange per interviewer
interview_slots	FK availability/interviewer; start/end/status; UNIQUE(interviewer,start,end); index(status,start)
interview_bookings	FK student/slot; attempt; active; state; idempotency_key UNIQUE; UNIQUE(slot_id); UNIQUE(student,attempt); partial UNIQUE active student
interview_results / retake_requests	Result UNIQUE booking; outcome/feedback/internal notes/submitted_by. Retake links prior booking and eligibility window.
malpractice_cases	FK student; reason/evidence; ALLEGED/VERIFIED/REJECTED; decided_by/at
badge_status_history	from/to state, actor, reason; append-only badge governance history
notifications / audit_logs	Student message/read time; actor/action/entity/json payload with immutable timestamp

E. Concurrency prevention - exact booking protocol

1. Client generates an idempotency key and calls only **book_interview_slot(student, slot, attempt, key)**. **2.** One PostgreSQL transaction executes **SELECT ... FOR UPDATE** on the slot. **3.** Under the lock, the server rechecks OPEN status, future start time, student eligibility/current state and absence of another active booking. **4.** It inserts the booking, conditionally updates the slot to BOOKED, advances student state, writes notification/audit, then commits. **5.** UNIQUE(slot_id), UNIQUE(idempotency_key), UNIQUE(student,attempt) and the partial active-booking index are final integrity barriers. If two requests race, the first commit wins; the second receives a stable BOOKING_CONFLICT/SLOT_UNAVAILABLE response and refreshes availability. Frontend checks improve UX but never decide correctness.

F. Product decisions

Decision	Implementation rationale
Eligibility	Derived from verified student skills against badge-specific required levels; no manual student override.
Availability	Availability is an interval; blocked time and booked slots are distinct. Existing bookings survive availability edits.
Results	Assigned interviewer selects one outcome, adds student feedback and optional private notes. Submission is auditable.
Retakes	RETAKE LATER creates RETAKE_REQUIRED; a later booking is a new numbered attempt, preserving history.
Malpractice	Two-step governance: allegation records evidence without changing badge; authorized verification alone transitions BADGE_EARNED to NULLIFIED.
Notifications	Booking confirmation, result, retake and nullification messages are action-triggered and persist in the student dashboard.
Operations	Funnel metrics, unbooked-eligible filter, interviewer workload, pending results and audit history make exceptions visible.
Demo/production	LocalStorage enables evaluator access without credentials. Supabase Auth + RLS + the included transaction RPC replace the demo adapter in production.

Validation and handoff

The prototype includes 10 students, 3 interviewers, multiple tracks, open/booked/blocked slots, PASS/FAIL/RETAKE/PENDING/NULLIFIED scenarios, role switching, responsive states, confirmations, audit events, lifecycle tests, SQL migration, README walkthrough and requirement checklist. Official GRIT references were used only for the Track -> Badge -> Skills/Levels -> Level Clearance Interview model; unspecified operating policy remains configurable.